Práctica N°1: Medicion de Corriente con ajuste de PWM visualizado en OLED

Índice

1. Objetivos **3**

2. Desarrollo **3**

2.1. Asignación de Pines en el STM32CubeMX 3

2.1.1. Pines para la Pantalla OLED (SPI1) 4

2.1.2. Pines para el Sensor de Corriente Hall (SPI2) 4

2.1.3. Pines para el PWM (TIM1) 5

2.1.4. Pines para el Driver de Motor 6

2.1.5. Pines para los Pulsadores de Control 6

2.1.6. Modo de Reseteo de Pines 7

2.2. Configuración de Periféricos en CubeMX 8

2.2.1. Configuración de SPI1 (Pantalla OLED) 8

2.2.2. Configuración de SPI2 (Sensor Hall) 9

2.2.3. Configuración de TIM1 (PWM) 10

2.2.4. Configuración de GPIOs 11

2.2.5. Configuración de Interrupciones Externas (EXTI) . . . 11

2.2.6. Configuración del Clock Configuration 12

2.3. Control de la Pantalla OLED (Librería SSD1351) 13

2.3.1. Explicación del código 15

2.3.2. Archivo de cabecera `ssd1351.h` 16

2.3.3. Creación de archivos adicionales para controladores externos 18

2.4. Estructura del archivo `main.c` 19

2.4.1. Librerías utilizadas 19

2.4.2. Definición de variables y constantes 20

2.5. Funciones implementadas 20

2.5.1. Interrupciones por pulsadores 21

2.5.2. Lectura del sensor de corriente 22

2.6. Desglose del archivo `main.c` 22
2.6.1. Librerías utilizadas 22
2.6.2. Definición de variables 23
2.6.3. Funciones auxiliares 23
2.6.4. Función `main()` y bucle infinito 24

3. Análisis del archivo `main.c` 26
3.1. Inclusión de librerías 26
3.2. Definiciones de variables y constantes 26
3.3. Funciones auxiliares 27
3.4. Función principal `main()` 28
3.5. Bucle `while(1)` 28

4. Resultados 29

1. Objetivos

Objetivo General

Implementar un sistema embebido en la tarjeta **STM32F407G-DISC1** con la **Discovery Shield**, que permita la medición de corriente mediante un sensor de pasillo o Hall, el control de una señal PWM para la regulación de un motor mediante pulsantes y la visualización de datos en una pantalla OLED.

Objetivos Específicos

- Configurar el entorno de desarrollo **STM32CubeIDE** y realizar la correcta inicialización de los periféricos necesarios (SPI, TIM, GPIO y UART).
- Adquirir y procesar la señal de un **sensor de corriente digital** a través de la interfaz SPI.
- Generar y controlar una señal **PWM** mediante los temporizadores del microcontrolador para regular la velocidad de un motor de corriente continua.
- Implementar la comunicación y control gráfico de una **pantalla OLED SSD1351** para mostrar la corriente medida y el estado del PWM.
- Integrar todos los módulos desarrollados en un único proyecto funcional sobre la placa STM32F407G-DISC1.
- Validar experimentalmente el correcto funcionamiento del sistema, verificando la relación entre la corriente medida, el control PWM y la visualización en la pantalla.

2. Desarrollo

2.1. Asignación de Pines en el STM32CubeMX

Una parte fundamental en la configuración de un proyecto en STM32CubeIDE es la asignación de pines dentro del archivo `.ioc`. Cada periférico del microcontrolador debe conectarse correctamente a los pines físicos de la tarjeta de desarrollo.

En esta práctica se requiere configurar los periféricos de la **pantalla OLED**, el **sensor de corriente Hall**, el **PWM para el control de motor**, el **driver de potencia** y los **pulsadores de control**.

2.1.1. Pines para la Pantalla OLED (SPI1)

La pantalla OLED **SSD1351** se conecta al microcontrolador mediante la interfaz **SPI1** y pines adicionales de control:

- **PB5**: SPI1_MOSI
- **PB4**: SPI1_MISO
- **PB3**: SPI1_SCK
- **PE5**: GPIO Output (Chip Select de la OLED)
- **PE4**: GPIO Output (RST de la OLED)
- **PB1**: GPIO Output (D/C de la OLED)
- **PD2**: GPIO Output (EN de la OLED)

Pines reseteados en esta etapa para evitar conflictos: **PE1, PB9, PB6, PB3, PD4, PC12, PC10**.

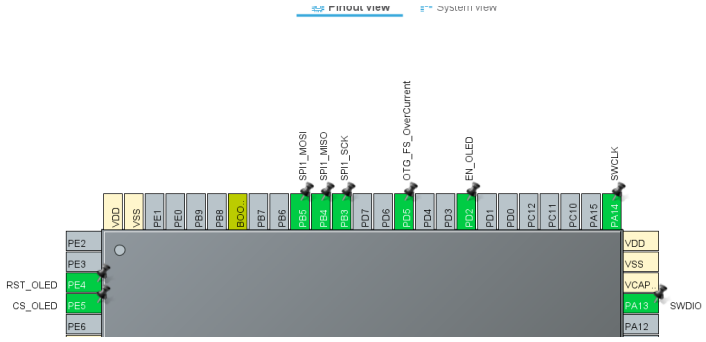


Figura 1: Configuración de pines para la pantalla OLED en el archivo `.ioc`.

2.1.2. Pines para el Sensor de Corriente Hall (SPI2)

El sensor de corriente digital (**TLI4970**) se comunica mediante la interfaz **SPI2**:

- **PB15**: SPI2_MOSI

- **PB14**: SPI2_MISO
- **PB13**: SPI2_SCK
- **PB12**: GPIO Output (Chip Select del sensor Hall)

Pines reseteados en esta etapa: **PA12, PA11, PA10, PA9, PC7**.

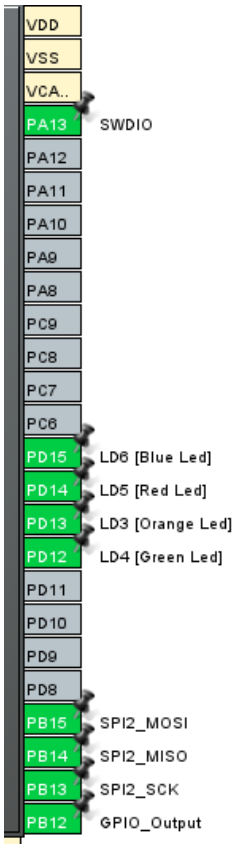


Figura 2: Configuración de pines para el sensor Hall en el archivo `.ioc`.

2.1.3. Pines para el PWM (TIM1)

El control PWM se genera con el temporizador **TIM1**, cuya salida principal se conecta en:

- **PE13**: TIM1_CH3 (PWM para el control del motor)

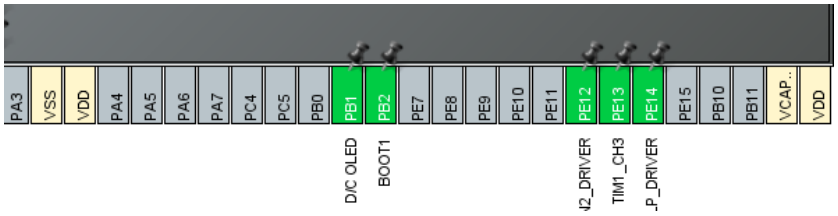


Figura 3: Configuración del pin de salida PWM en el archivo `.ioc`.

2.1.4. Pines para el Driver de Motor

El driver **TB6549FG** requiere tres señales de control adicionales:

- **PE14**: GPIO Output (SLP, habilitación del driver)
- **PE12**: GPIO Output (IN2)
- **PC2**: GPIO Output (IN1)

2.1.5. Pines para los Pulsadores de Control

Se emplean dos pulsadores conectados a entradas GPIO_EXIT configuradas como interrupciones externas:

- **PA1**: GPIO External Interrupt (disminuye el valor del PWM)
- **PA2**: GPIO External Interrupt (aumenta el valor del PWM)

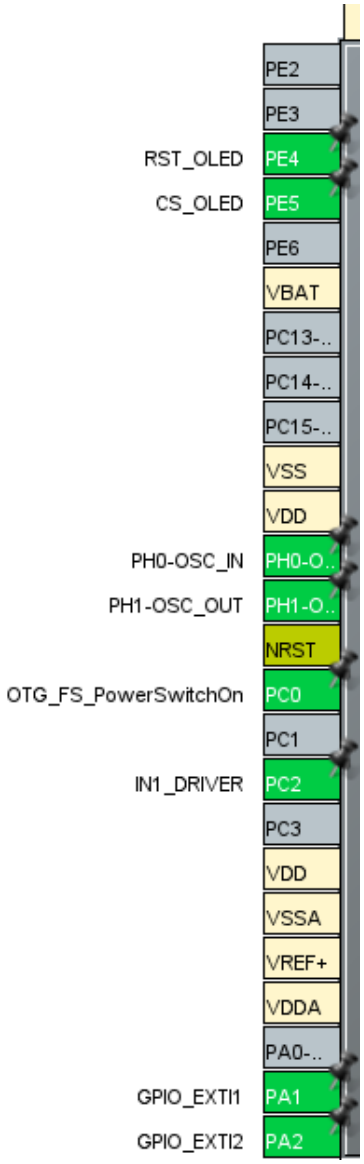


Figura 4: Configuración de pines para los pulsadores de control del PWM.

2.1.6. Modo de Reseteo de Pines

En CubeMX, cada pin del microcontrolador puede asignarse a una función periférica, a un GPIO o quedar en estado de **Reset State**. El **modo Reset** significa que el pin permanece en su estado por defecto (alta impedancia o sin función asignada), evitando interferencias con otros periféricos.

Para resetear un pin en CubeMX:

1. Seleccionar el pin correspondiente en el diagrama del microcontrolador.

2. En el menú emergente, escoger la opción **Reset State**.

3. Verificar en la lista de pines (Pinout & Configuration) que el pin ya no está asignado a ninguna función activa.

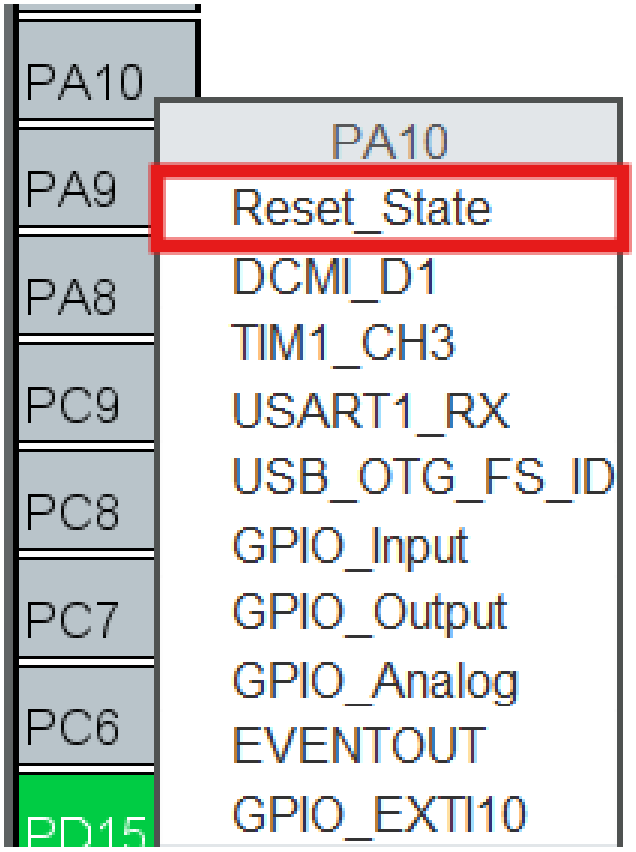


Figura 5: Proceso de reseteo de pines en CubeMX.

2.2. Configuración de Periféricos en CubeMX

Una vez asignados los pines, el siguiente paso consiste en configurar los periféricos en la pestaña **Configuration** del archivo `.ioc`. Esta etapa permite definir parámetros esenciales como la velocidad de comunicación, los modos de operación y la inicialización de cada módulo.

2.2.1. Configuración de SPI1 (Pantalla OLED)

La pantalla OLED basada en el controlador SSD1351 se comunica a través de la interfaz **SPI1**.

Los parámetros recomendados son:

- **Mode**: Full-Duplex Master.
- **Prescaler**: 8 (para obtener una frecuencia de reloj adecuada a la OLED).
- **Data Size**: 8 Bits.
- **Clock Polarity (CPOL)**: Low.
- **Clock Phase (CPHA)**: 1 Edge.
- **First Bit**: MSB First.

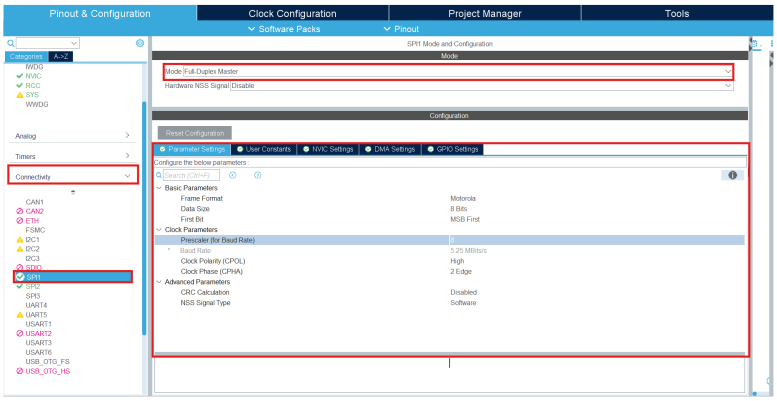


Figura 6: Configuración de SPI1 para la comunicación con la pantalla OLED.

2.2.2. Configuración de SPI2 (Sensor Hall)

El sensor de corriente TLI4970 emplea la interfaz **SPI2**. Su configuración es similar a SPI1, pero puede requerir un prescaler más bajo dependiendo de la velocidad de lectura deseada.

- **Mode**: Full-Duplex Master.
- **Prescaler**: 16.
- **Data Size**: 8 Bits.
- **CPOL**: High.
- **CPHA**: 2 Edge.
- **First Bit**: MSB First.

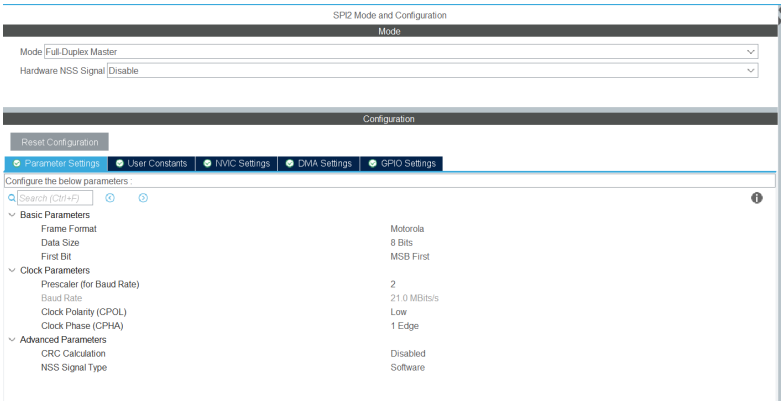


Figura 7: Configuración de SPI2 para el sensor de corriente Hall.

2.2.3. Configuración de TIM1 (PWM)

El **TIM1** se utiliza para generar la señal PWM que controla la velocidad del motor.

Parámetros a configurar:

- **Channel 3**: PWM Generation CH3 (activo en PE13).
- **Prescaler**: 83 (para dividir la frecuencia de reloj de 84 MHz).
- **Counter Period**: 999 (resolución de 1000 pasos, corresponde a 1 kHz de frecuencia de PWM).
- **PWM Mode**: PWM Mode 1.
- **Pulse**: valor inicial de duty cycle (ej. 100).

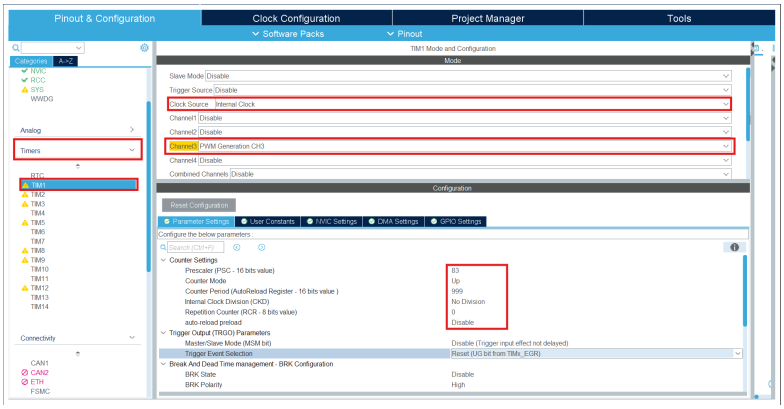


Figura 8: Configuración de TIM1 para la generación de la señal PWM.

2.2.4. Configuración de GPIOs

Los pines de control de la OLED y del driver de motor deben configurarse como **GPIO Output**:

- **OLED**: PE5 (CS), PE4 (RST), PB1 (D/C), PD2 (EN).
- **Driver de Motor**: PE14 (SLP), PE12 (IN2), PC2 (IN1).

Modo de configuración:

- Mode: Output Push-Pull.
- Pull-up/Pull-down: No pull.
- Maximum Output Speed: High.

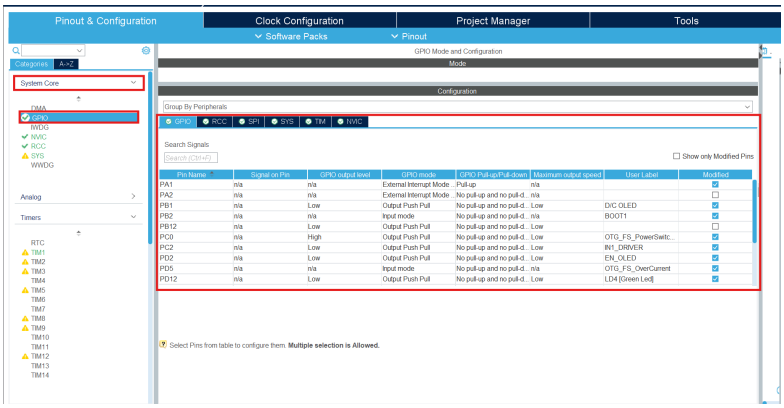


Figura 9: Configuración de GPIOs de control para la OLED y el driver de motor.

2.2.5. Configuración de Interrupciones Externas (EXTI)

Los pulsadores de control del PWM se configuran en modo interrupción:

- **PA1**: External Interrupt, Rising Edge.
- **PA2**: External Interrupt, Rising Edge.

Esto permitirá modificar el duty cycle al presionar los botones.

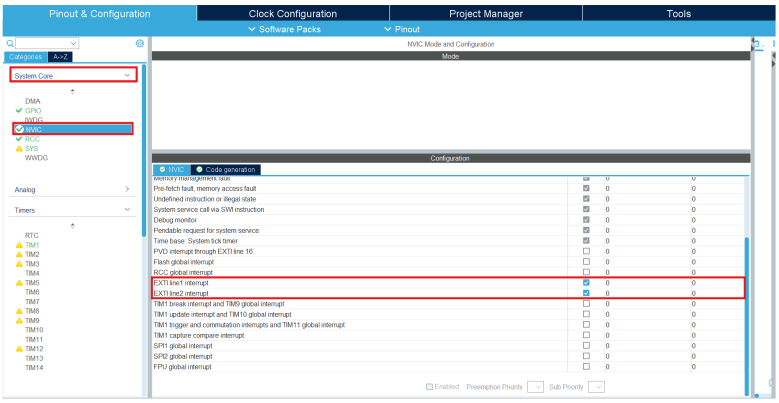


Figura 10: Configuración de interrupciones externas para los pulsadores.

2.2.6. Configuración del Clock Configuration

Finalmente, es necesario verificar la pestaña **Clock Configuration** en CubeMX, la cual permite ajustar la velocidad del sistema y de los periféricos. Para esta práctica se recomienda mantener el microcontrolador trabajando a **84 MHz**, frecuencia suficiente para el manejo simultáneo de SPI, PWM y demás periféricos.

Los pasos son los siguientes:

1. Ingresar a la pestaña **Clock Configuration**.
2. Seleccionar como fuente principal el **HSE (High Speed External oscillator)** de 8 MHz presente en la tarjeta STM32F407G-DISC1.
3. Configurar el **PLL** de la siguiente forma:
   - PLL_M = 8
   - PLL_N = 336
   - PLL_P = 4
   - PLL_Q = 7
4. Verificar que la frecuencia del **SYSCLK** sea de 84 MHz.
5. Confirmar que las frecuencias derivadas (APB1, APB2, AHB) están dentro de los valores recomendados por el fabricante:
   - AHB: 84 MHz
   - APB1: 42 MHz
   - APB2: 84 MHz

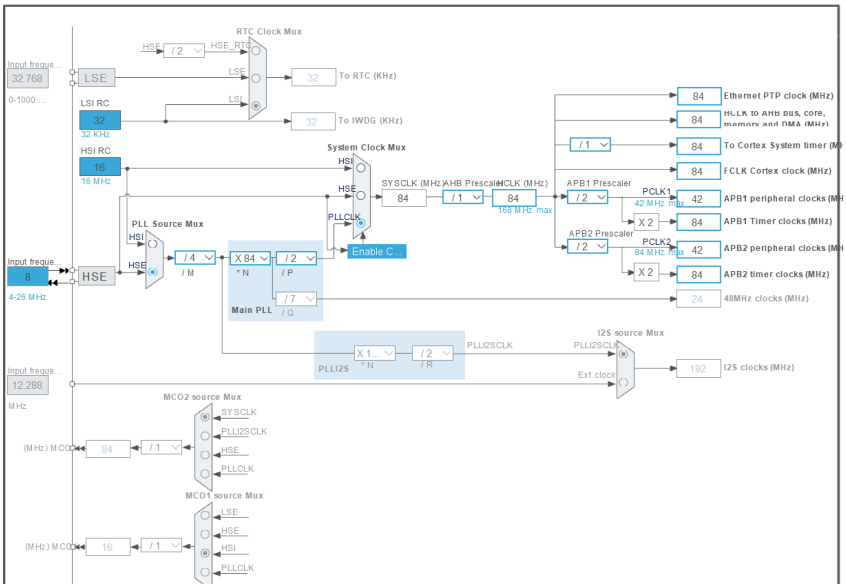


Figura 11: Configuración de reloj en CubeMX para trabajar con un SYSCLK de 84 MHz.

De esta manera, se asegura que los temporizadores (ejemplo: TIM1 para PWM) y las interfaces de comunicación (SPI1 y SPI2) operen con las frecuencias correctas y estables, evitando errores en la transmisión o en el cálculo de periodos.

2.3. Control de la Pantalla OLED (Librería SSD1351)

Para manejar la pantalla OLED integrada en la práctica se desarrolló una librería propia, contenida en el archivo `ssd1351.c`. Esta librería implementa las funciones de inicialización, limpieza de pantalla, escritura de pixeles y renderizado de texto en la pantalla mediante la interfaz **SPI1**.

A continuación se muestra el código completo del archivo, con un formato que permite visualizarlo de manera similar a como aparece en el entorno de CubeIDE:

```
#include <ssd1351.h>
#include "main.h"

extern SPI_HandleTypeDef hspi1;
void SSD1351_DrawPixel(uint8_t x, uint8_t y, uint16_t color);

#define DC_GPIO_Port GPIOB
#define DC_Pin GPIO_PIN_1
#define CS_GPIO_Port GPIOE
#define CS_Pin GPIO_PIN_5
```

```
#define RST_GPIO_Port GPIOE
#define RST_Pin GPIO_PIN_4

/* --- Funciones internas para enviar comandos y datos --- */
static void SSD1351_SendCommand(uint8_t cmd) {
    HAL_GPIO_WritePin(DC_GPIO_Port, DC_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, &cmd, 1, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
}

static void SSD1351_SendData(uint8_t *data, uint16_t size) {
    HAL_GPIO_WritePin(DC_GPIO_Port, DC_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, data, size, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
}

/* --- Inicializacion del display --- */
void SSD1351_Init(void) {
    HAL_GPIO_WritePin(RST_GPIO_Port, RST_Pin, GPIO_PIN_RESET)
        ;
    HAL_Delay(20);
    HAL_GPIO_WritePin(RST_GPIO_Port, RST_Pin, GPIO_PIN_SET);
    HAL_Delay(20);

    SSD1351_SendCommand(0xAE); // Display off
    /* ... Configuracion completa del display ... */
    SSD1351_SendCommand(0xAF); // Display on
}

/* --- Limpiar la pantalla --- */
void SSD1351_Clear(void) {
    SSD1351_SendCommand(0x15);
    SSD1351_SendData((uint8_t[]){0x00, 0x7F}, 2);
    SSD1351_SendCommand(0x75);
    SSD1351_SendData((uint8_t[]){0x00, 0x7F}, 2);
    SSD1351_SendCommand(0x5C);
    uint8_t black[] = {0x00, 0x00};
    for (uint16_t i = 0; i < 128 * 128; i++) {
        SSD1351_SendData(black, 2);
    }
}

/* --- Dibujar un pixel individual --- */
void SSD1351_DrawPixel(uint8_t x, uint8_t y, uint16_t color)
    {
    if (x >= SSD1351_WIDTH || y >= SSD1351_HEIGHT) return;
    SSD1351_SendCommand(0x15);
```

```
58      SSD1351_SendData((uint8_t[]){x, x}, 2);
59      SSD1351_SendCommand(0x75);
60      SSD1351_SendData((uint8_t[]){y, y}, 2);
61      SSD1351_SendCommand(0x5C);
62      uint8_t data[2] = {color >> 8, color & 0xFF};
63      SSD1351_SendData(data, 2);
64  }
65
66  /* --- Fuente de caracteres 5x8 --- */
67  const uint8_t font5x8[][5] = {
68     /* ... definicion completa de caracteres ... */
69  };
70
71  /* --- Funcion para dibujar texto en pantalla --- */
72  void SSD1351_DrawText(uint8_t x, uint8_t y, const char *text,
73                        uint16_t color, uint16_t bg, uint8_t
                             scale) {
74      while (*text) {
75          char c = *text++;
76          if (c < 32 || c > 127) c = '?';
77          const uint8_t *bitmap = font5x8[c - 32];
78          /* ... renderizado pixel por pixel ... */
79      }
80  }
```

Listing 1: Implementación de la librería SSD1351 en el archivo `ssd1351.c`

2.3.1. Explicación del código

El archivo `ssd1351.c` se organiza en varias secciones bien diferenciadas:

- **Definición de pines y dependencias**: Se definen los pines de control (`DC`, `CS`, `RST`) que permiten la comunicación con el OLED mediante SPI1.
- **Funciones internas de envío**: `SSD1351_SendCommand()` y `SSD1351_SendData()` encapsulan la transmisión por SPI, diferenciando entre comandos y datos.
- **Inicialización**: La función `SSD1351_Init()` configura el controlador del display paso a paso según su hoja de datos, asegurando que arranque en modo correcto.
- **Funciones gráficas**:
  - `SSD1351_Clear()` limpia la pantalla con un color de fondo.

    - `SSD1351_DrawPixel()` permite dibujar un pixel individual en formato `RGB565`.

- **Fuente de caracteres**: Se incluye una tabla de caracteres 5x8 para poder renderizar texto básico.
- **Dibujo de texto**: `SSD1351_DrawText()` recorre la cadena recibida, consultando la fuente y escribiéndola pixel por pixel en pantalla, soportando además escalado.

2.3.2. Archivo de cabecera `ssd1351.h`

El archivo de cabecera define las constantes, prototipos de funciones y librerías necesarias para controlar la pantalla OLED SSD1351. Este archivo debe guardarse en la carpeta `Core/Inc` y será incluido posteriormente en el `main.c`.

A continuación, se muestra el contenido del archivo:

```c
#ifndef __SSD1351_H__
#define __SSD1351_H__

#include "stm32f4xx_hal.h"
#include <stdint.h>

// Resolucion de la pantalla OLED SSD1351
#define SSD1351_WIDTH  128
#define SSD1351_HEIGHT 128

// Prototipos de funciones para el control de la pantalla
void SSD1351_Init(void);
void SSD1351_Clear(void);
void SSD1351_DrawText(uint8_t x, uint8_t y, const char *text,
                      uint16_t color, uint16_t bg, uint8_t
                         scale);

#endif
```

Listing 2: Código del archivo `ssd1351.h`

Explicación de los elementos principales:

- `#ifndef ... #define ... #endif`: Esta construcción se denomina *include guard*, y evita que el archivo sea incluido más de una vez en el proyecto, lo cual podría generar errores de compilación.
- `#include "stm32f4xx_hal.h"`: Permite el uso de las funciones de la librería HAL específicas del microcontrolador STM32F4.
- `#include <stdint.h>`: Incluye definiciones de tipos de datos estándar como `uint8_t` o `uint16_t`, muy usados en programación embebida.
- `#define SSD1351_WIDTH 128` y `#define SSD1351_HEIGHT 128`: Constantes que definen la resolución de la pantalla OLED (128x128 píxeles).
- `SSD1351_Init()`: Función para inicializar la pantalla OLED (se implementa en `ssd1351.c`).
- `SSD1351_Clear()`: Borra el contenido actual de la pantalla y la rellena con un color de fondo.
- `SSD1351_DrawText(...)`: Permite escribir texto en la pantalla, especificando posición (`x, y`), color de texto, color de fondo y tamaño de escala.

Este archivo actúa como la *interfaz pública* del controlador, ya que otros archivos del proyecto (como el `main.c`) podrán usar estas funciones simplemente incluyendo la línea:

```
#include "ssd1351.h"
```

2.3.3. Creación de archivos adicionales para controladores externos

En esta práctica no solo se utilizarán los archivos que CubeMX genera de forma automática, sino que también será necesario incluir controladores adicionales, como el de la pantalla OLED SSD1351. Estos controladores deben organizarse de forma adecuada dentro del proyecto para que puedan ser incluidos en el `main.c` y utilizados sin problemas.

Los pasos para crear un nuevo archivo en STM32CubeIDE son los siguientes:

1. En el explorador de proyectos (panel izquierdo), ubicar la carpeta llamada `Core/Src` para los archivos `.c`, y la carpeta `Core/Inc` para los archivos `.h`.

2. Hacer clic derecho sobre la carpeta correspondiente y seleccionar:

   - `New` → `Source File` para un archivo con extensión `.c`.
   - `New` → `Header File` para un archivo con extensión `.h`.

3. Asignar el nombre del archivo, por ejemplo:

   - `ssd1351.c` dentro de la carpeta `Core/Src`.
   - `ssd1351.h` dentro de la carpeta `Core/Inc`.

4. En el archivo de cabecera `ssd1351.h` se definen las funciones, constantes y prototipos necesarios para controlar la pantalla OLED.

5. En el archivo `ssd1351.c` se implementa la lógica de dichas funciones (inicialización, limpieza de pantalla, escritura de texto, etc.).

6. Finalmente, para poder utilizar este controlador en el `main.c`, se debe incluir la cabecera con:

   ```
   #include "ssd1351.h"
   ```

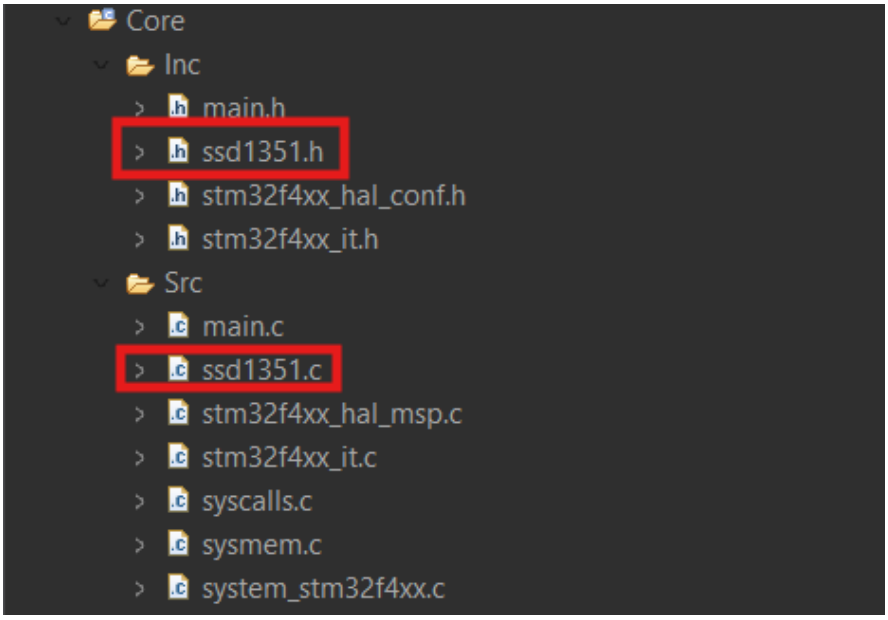


Figura 12: Ubicación de archivos adicionales en STM32CubeIDE.

2.4. Estructura del archivo `main.c`

El archivo principal del proyecto generado por CubeMX (`main.c`) constituye el núcleo del programa, ya que en él se inicializan los periféricos configurados, se definen las variables globales, se implementan funciones auxiliares y se establece la lógica principal dentro del bucle infinito `while(1)`. A continuación, se desglosan las diferentes partes del código.

2.4.1. Librerías utilizadas

En la parte inicial se incluyen tanto las librerías básicas de CubeMX como las adicionales requeridas por la práctica.

```
#include "main.h"
#include <stdio.h>
#include <string.h>
#include "ssd1351.h"
```

Listing 3: Inclusión de librerías en `main.c`

- `main.h`: Cabecera generada automáticamente por CubeMX con las definiciones de pines y periféricos.
- `stdio.h` y `string.h`: Librerías estándar de C para funciones de manejo de cadenas y formato.

- `ssd1351.h`: Archivo de cabecera creado por el usuario para controlar la pantalla OLED.

2.4.2. Definición de variables y constantes

En esta sección se declaran los manejadores de periféricos, variables globales y constantes necesarias para el control del sistema.

```
SPI_HandleTypeDef hspi1;
SPI_HandleTypeDef hspi2;
TIM_HandleTypeDef htim1;

#define BLACK   0x0000
#define WHITE   0xFFFF
#define RED     0xF800
#define GREEN   0x07E0
#define BLUE    0x001F
#define YELLOW  0xFFE0

uint32_t last_press_time = 0;
volatile uint8_t step_count = 0;
const uint8_t duty_steps[6] = {0, 20, 40, 60, 80, 90};
```

Listing 4: Variables y constantes en `main.c`

- `hspi1, hspi2`: Manejadores para las interfaces SPI, usados en la OLED y en el sensor de corriente.
- `htim1`: Manejador para el temporizador encargado de generar la señal PWM.
- Constantes de colores: Representan los valores RGB565 usados para la pantalla OLED.
- `last_press_time`: Variable de control para implementar antirrebote en los botones.
- `step_count`: Índice de paso para determinar el ciclo de trabajo (% de PWM).
- `duty_steps`: Arreglo con los valores discretos de PWM permitidos en la práctica.

2.5. Funciones implementadas

En la práctica se definen varias funciones auxiliares para el control del PWM y la lectura de corriente desde el sensor Hall.

Control de PWM

```
void Set_PWM_Duty(uint8_t percent) {
    if (percent > 100) return;
    uint32_t pulse = (percent * (__HAL_TIM_GET_AUTORELOAD(&
       htim1) + 1)) / 100;
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse);
}
```

Esta función recibe como argumento un porcentaje y ajusta el registro de comparación del temporizador para variar el ciclo de trabajo del PWM.

2.5.1. Interrupciones por pulsadores

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (HAL_GetTick() - last_press_time < 50) return;

    if (GPIO_Pin == GPIO_PIN_1 && step_count < 5) {
        step_count++;
    } else if (GPIO_Pin == GPIO_PIN_2 && step_count > 0) {
        step_count--;
    }
    Set_PWM_Duty(duty_steps[step_count]);
    last_press_time = HAL_GetTick();
}
```

Aquí se controla el incremento o decremento del PWM mediante botones conectados a los pines `PA1` y `PA2`, incluyendo un filtro de software para el antirrebote.

2.5.2. Lectura del sensor de corriente

```
int16_t Read_Current_Signed(void) {
    uint8_t tx_dummy[2] = {0x00, 0x00};
    uint8_t rx_data[2] = {0};

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_RESET); // CS LOW
    HAL_SPI_TransmitReceive(&hspi2, tx_dummy, rx_data, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_SET);   // CS HIGH

    uint16_t raw = ((uint16_t)rx_data[0] << 8) | rx_data[1];
    int16_t current_raw = raw & 0x1FFF;
    if (current_raw & 0x1000) {
        current_raw |= 0xE000; // Extiende signo
    }
    return current_raw;
}
```

La función lee dos bytes por SPI desde el sensor Hall de corriente y procesa los datos para devolver un valor con signo.

2.6. Desglose del archivo `main.c`

En esta sección se explicará la estructura del archivo principal del proyecto, mostrando de manera ordenada el uso de librerías, variables globales, funciones auxiliares, la función `main()` y el bucle infinito `while`.

2.6.1. Librerías utilizadas

Las librerías incluidas en este proyecto permiten acceder a las funciones de la HAL (Hardware Abstraction Layer) de STM32 y a utilidades estándar como `stdio.h` para depuración.

```
#include "main.h"
#include <stdio.h>
```

Listing 5: Librerías incluidas en `main.c`

- `main.h`: Archivo de cabecera generado por CubeMX que contiene la configuración de pines, macros y prototipos.
- `stdio.h`: Usado para redirigir funciones como `printf()` hacia la UART.

2.6.2. Definición de variables

Se definen estructuras de la HAL para manejar periféricos, así como variables para el control del PWM y el sensor de corriente.

```
SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart2;
TIM_HandleTypeDef htim1;

uint32_t last_press_time = 0;  // Antirrebote
volatile uint8_t step_count = 0;

// Escalones de duty cycle en %
const uint8_t duty_steps[6] = {0, 20, 40, 60, 80, 90};
```

Listing 6: Variables globales

- `hspi1`, `huart2`, `htim1`: Estructuras de la HAL que almacenan la configuración activa de SPI, UART y Timer.
- `last_press_time`: Control de tiempo para evitar rebotes en el botón.
- `step_count`: Índice del nivel de PWM actual.
- `duty_steps`: Arreglo con los valores de duty cycle disponibles.

2.6.3. Funciones auxiliares

El código define varias funciones que facilitan el manejo de periféricos.

```
void Set_PWM_Duty(uint8_t percent) {
    if (percent > 100) return;
    uint32_t pulse = (percent * (__HAL_TIM_GET_AUTORELOAD(&
       htim1) + 1)) / 100;
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, pulse);
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_1) {
        if (HAL_GetTick() - last_press_time > 50) { //
           Antirrebote
            step_count = (step_count + 1) % 6;
            Set_PWM_Duty(duty_steps[step_count]);
            last_press_time = HAL_GetTick();
        }
    }
}

```

```
17 uint16_t Read_Current_Raw(void) {
18     uint8_t tx_dummy[2] = {0x00, 0x00};
19     uint8_t rx_data[2] = {0};
20 
21     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_RESET); //
           CS LOW
22     HAL_SPI_TransmitReceive(&hspi1, tx_dummy, rx_data, 2,
           HAL_MAX_DELAY);
23     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);   //
           CS HIGH
24 
25     uint16_t raw = ((uint16_t)rx_data[0] << 8) | rx_data[1];
26     return raw & 0x0FFF;  // 12 bits validos
27 }
28 
29 // Redirigir printf al puerto serie
30 int __io_putchar(int ch) {
31     HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1,
           HAL_MAX_DELAY);
32     return ch;
33 }
```

Listing 7: Funciones auxiliares

- `Set_PWM_Duty`: Ajusta el ciclo útil del PWM en base a un valor porcentual.
- `HAL_GPIO_EXTI_Callback`: Rutina de interrupción para el pulsador que incrementa el duty cycle.
- `Read_Current_Raw`: Lee 12 bits crudos del sensor de corriente a través de SPI.
- `__io_putchar`: Redirige `printf()` hacia la UART2.

2.6.4. Función `main()` y bucle infinito

El `main()` realiza la inicialización de periféricos, configura el reloj del sistema y entra en el bucle `while(1)`, donde ocurre la lógica principal.

```
1 int main(void)
2 {
3   HAL_Init();                 // Inicializacion de la HAL
4   SystemClock_Config();       // Configuracion de reloj
5   MX_GPIO_Init();
6   MX_SPI1_Init();
7   MX_USART2_UART_Init();
```

```
8   MX_TIM1_Init();
9
10  // Iniciar PWM en canal 1
11  HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
12  Set_PWM_Duty(0);  // Arranca en 0%
13
14  while (1)
15  {
16      uint32_t suma_raw = 0;
17      for (int i = 0; i < 10; i++) {
18          suma_raw += Read_Current_Raw();
19          HAL_Delay(10);
20      }
21
22      uint16_t raw_promedio = suma_raw / 10;
23      float current_mA = (raw_promedio - 2049) * 25.0f;
24
25      // Imprimir resultados
26      printf("Corriente Promedio: %.2f mA | Raw: %u | PWM
            Duty: %u%%\r\n",
27              current_mA, raw_promedio, duty_steps[step_count
                ]);
28
29      // Ajuste automatico de PWM
30      if (current_mA > 100.0f && step_count < 5) {
31          step_count++;
32          Set_PWM_Duty(duty_steps[step_count]);
33          printf("PWM aumentado a %u%% por corriente alta.\r\
                n", duty_steps[step_count]);
34      }
35
36      HAL_Delay(1000);  // 1 segundo entre mediciones
37  }
38 }
```

Listing 8: Fragmento de la función `main`

- Se inicializan todos los periféricos generados por CubeMX (`GPIO`, `SPI1`, `UART2`, `TIM1`).
- El PWM inicia en 0 % y se incrementa por medio del pulsador o automáticamente si la corriente medida supera un umbral.
- Cada segundo se promedian 10 muestras del sensor y se calcula la corriente.
- Los datos son enviados por UART para monitoreo en consola.

3. Análisis del archivo `main.c`

En este apartado se presenta el desglose del archivo principal del proyecto, `main.c`. El objetivo es comprender la lógica detrás de la inicialización, la organización de funciones y el bucle principal que gobierna la práctica.

3.1. Inclusión de librerías

El archivo comienza incluyendo tanto librerías estándar como específicas del microcontrolador.

```
#include "main.h"
#include <stdio.h>
#include <string.h>
#include "ssd1351.h"
```

- `main.h`: Generado automáticamente por CubeMX, contiene definiciones de pines y prototipos básicos.
- `stdio.h` y `string.h`: Librerías estándar de C para manejo de cadenas y funciones de formato.
- `ssd1351.h`: Controlador externo desarrollado para la pantalla OLED.

3.2. Definiciones de variables y constantes

En esta sección se encuentran las variables globales, estructuras de configuración de periféricos y constantes de uso general.

```
SPI_HandleTypeDef hspi1;
SPI_HandleTypeDef hspi2;
TIM_HandleTypeDef htim1;

#define BLACK   0x0000
#define WHITE   0xFFFF
#define RED     0xF800
#define GREEN   0x07E0
#define BLUE    0x001F
#define YELLOW  0xFFE0

uint32_t last_press_time = 0;  // Antirrebote
volatile uint8_t step_count = 0;
const uint8_t duty_steps[6] = {0, 20, 40, 60, 80, 90};
```

- **Colores**: definidos en formato RGB565, compatibles con la OLED.

- **last_press_time**: almacena la última pulsación para implementar antirrebote.
- **step_count**: índice que controla el nivel de PWM actual.
- **duty_steps**: tabla de posibles valores de ciclo de trabajo (%) para el PWM.

3.3. Funciones auxiliares

Se implementan funciones que encapsulan comportamientos específicos como ajuste de PWM, interrupciones y lectura del sensor de corriente.

Ajuste de ciclo de trabajo PWM

```
void Set_PWM_Duty(uint8_t percent) {
    if (percent > 100) return;
    uint32_t pulse = (percent * (__HAL_TIM_GET_AUTORELOAD(&
        htim1) + 1)) / 100;
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, pulse);
}
```

Esta función calcula el valor del registro de comparación para fijar el ciclo útil de la señal PWM.

Interrupción por pulsadores

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (HAL_GetTick() - last_press_time < 50) return;  //
        Antirrebote

    if (GPIO_Pin == GPIO_PIN_1 && step_count < 5) {
        step_count++;
    } else if (GPIO_Pin == GPIO_PIN_2 && step_count > 0) {
        step_count--;
    }

    Set_PWM_Duty(duty_steps[step_count]);
    last_press_time = HAL_GetTick();
}
```

Permite incrementar o decrementar el ciclo útil mediante interrupciones de los pines `PA1` y `PA2`.

Lectura del sensor de corriente

Se implementan funciones para comunicarse con el sensor a través del bus SPI2.

```
uint16_t Read_Current_Raw(void) { ... }
int16_t Read_Current_Signed(void) { ... }
```

Ambas realizan una transacción SPI y reconstruyen el valor de 13 bits que entrega el sensor Hall de corriente.

3.4. Función principal `main()`

La función `main` organiza la ejecución del programa. Primero inicializa el hardware, luego configura periféricos y finalmente entra en el bucle infinito.

1. Inicialización de HAL y reloj del sistema.
2. Configuración de periféricos: GPIO, SPI1, SPI2, TIM1.
3. Encendido del PWM en el canal 3 de TIM1.
4. Activación de pines de control para el driver de motor y la OLED.
5. Inicialización de la pantalla OLED y limpieza de contenido.

3.5. Bucle `while(1)`

El ciclo infinito contiene la lógica de la práctica:

```c
while (1) {
    uint32_t suma_raw = 0;
    for (int i = 0; i < 10; i++) {
        suma_raw += Read_Current_Raw();
        HAL_Delay(10);
    }

    uint16_t raw_prom = suma_raw / 10;
    float current_mA = (raw_prom - 2048) * 25.0f;

    snprintf(buffer_current, sizeof(buffer_current), "I: %.1
       fmA", current_mA);
    snprintf(buffer_duty, sizeof(buffer_duty), "D: %u%%",
       duty_steps[step_count]);

    SSD1351_Clear();
    SSD1351_DrawText(10, 10, buffer_current, BLUE, BLACK, 2);
    SSD1351_DrawText(10, 30, buffer_duty, BLUE, BLACK, 2);

    if (current_mA > 100.0f && step_count < 5) {
        step_count++;
        Set_PWM_Duty(duty_steps[step_count]);
    }

    HAL_Delay(1000);
}
```

Flujo general:

1. Se promedian 10 lecturas consecutivas del sensor para reducir ruido.
2. Se convierte la lectura cruda en corriente (mA).
3. Se actualiza la pantalla OLED con el valor de corriente y el ciclo de trabajo.
4. Se implementa una condición de protección: si la corriente supera 100 mA, se aumenta automáticamente el ciclo útil.
5. Se repite el proceso cada segundo.

4. Resultados

En esta sección se presentan los resultados obtenidos al implementar el sistema de medición de corriente, control de PWM y visualización en pantalla OLED utilizando la tarjeta **STM32F407G-DISC1** y la Discovery Shield.

Se configuró el generador PWM con un ciclo de trabajo del **90 %**, lo que permitió alcanzar una velocidad elevada en el motor y, en consecuencia, un mayor consumo de corriente. Estos valores fueron correctamente medidos por el sensor de corriente Hall y mostrados en la pantalla OLED, confirmando el correcto funcionamiento del sistema integrado.

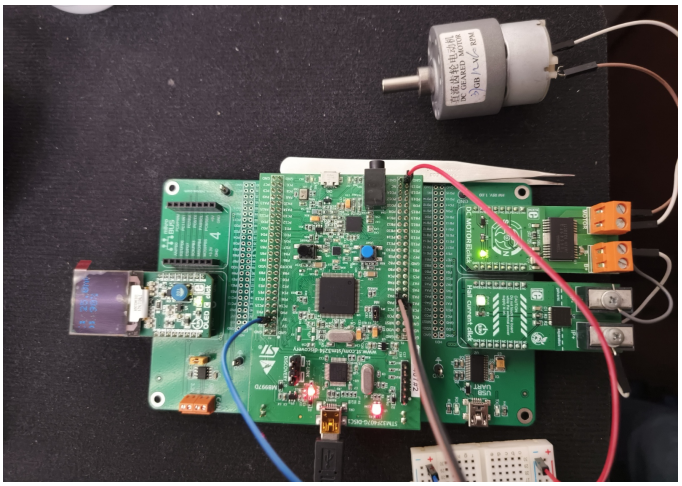


Figura 13: Visualización en pantalla OLED y comportamiento del motor con un ciclo de trabajo del 90 %.

El resultado evidencia que el sistema responde de manera adecuada al incremento en el ciclo de trabajo, mostrando en la pantalla OLED el valor de la corriente consumida y reflejando en el motor un régimen de operación de alta velocidad.